Is the model any good?

- **Every test score is a claim:** this is how the model will do on data it has not seen
- **The claim is easy to get wrong on environmental data,** and the mistake makes the model look better than it is
- **Three ideas carry the week:** a baseline, a split that matches the application, and settings chosen on data the score never sees
- **Then one more question:** not whether the model is good, but how wrong a particular prediction might be

A baseline

climatology, persistence, or the mean. No score means anything on its own

↓

A split that matches the application

many splits, not one, and folds that respect space and time

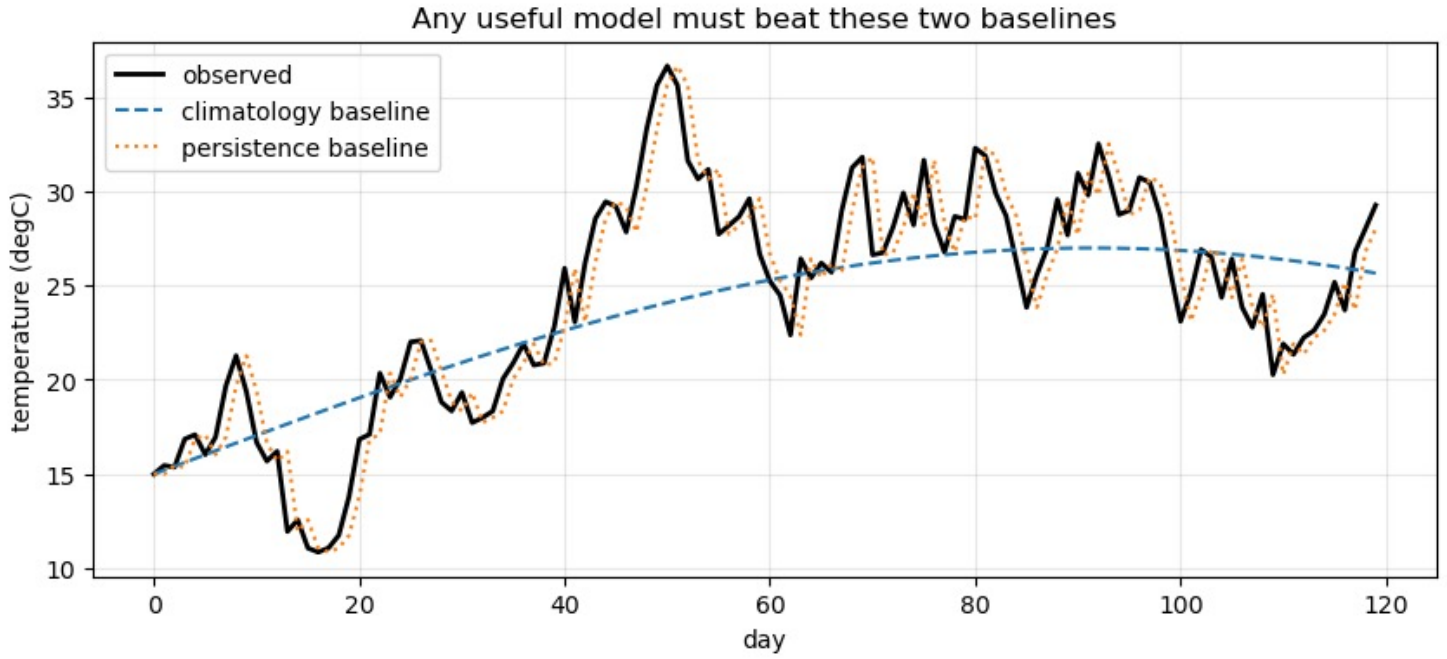
↓

Settings chosen without cheating

validation data or cross-validation chooses them; the test set is scored once

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

Always compare against a baseline

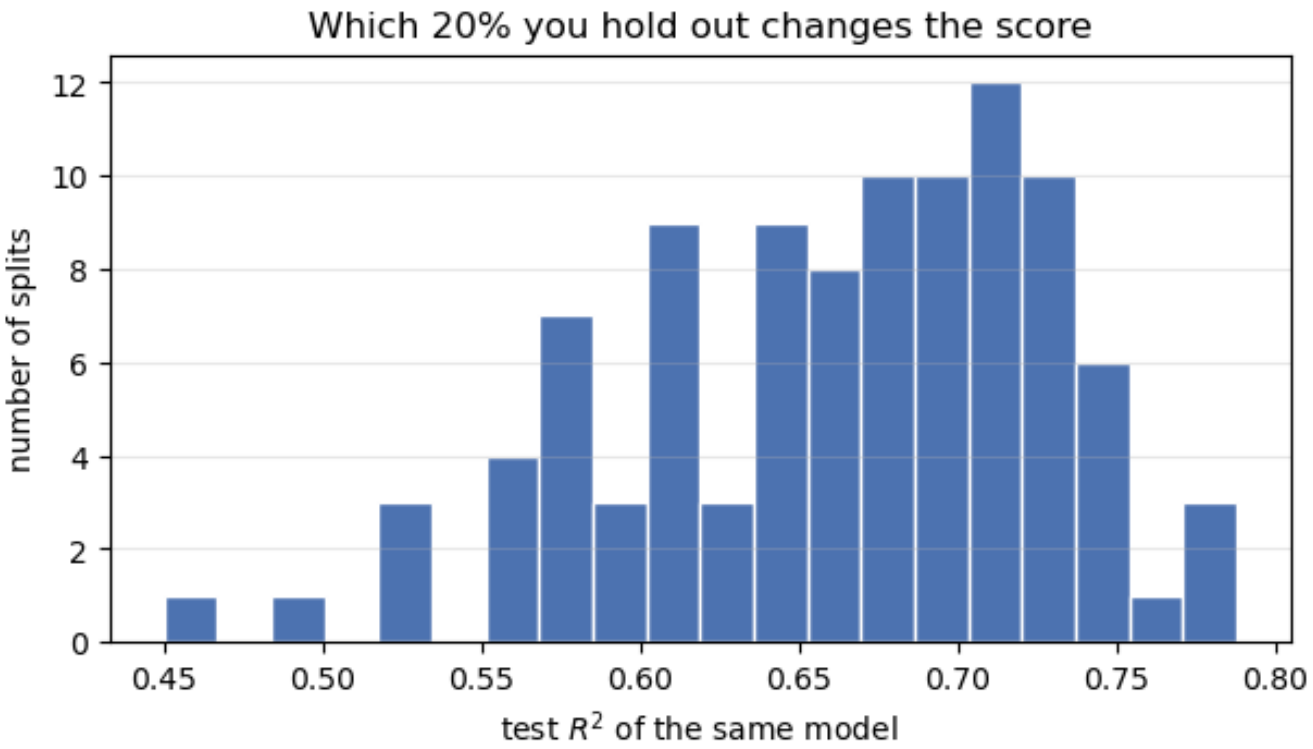


Baseline	R^2
Climatology: the seasonal mean	0.87
Persistence: yesterday's value	0.96

- **Climatology:** the long-term mean for that place and time of year
- **Persistence:** the most recent observation
- **Both are hard to beat,** because weather carries over from day to day. That is the point
- **A skill score without its baseline is not a result**

Figure from the Week 6 lecture notes

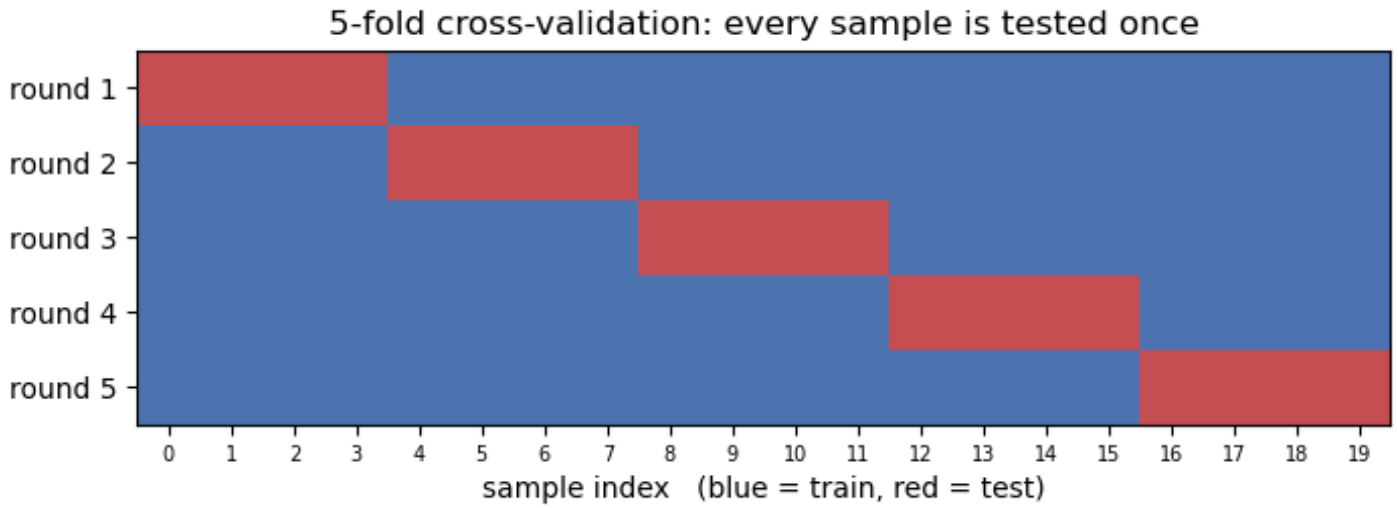
One hold-out score is one noisy number



- **Same model, same 300 samples, 100 random 80/20 splits**
- **Test R^2 runs from 0.45 to 0.79.** Which 60 samples you held out decided the score
- **A single hold-out is one draw from this histogram,** and you do not know where in it you landed
- **Small data sets make it worse,** and station records, field campaigns, and model ensembles are small

Figure from the Week 6 lecture notes

K-fold cross-validation



What it buys: all the data tested, most of the data trained on in every round, and a spread.
What it costs: k fits instead of one.

- **Cut the data into k folds.** Hold out each in turn, fit on the rest, score on the held-out fold
- **Every sample is tested once** and trained on k − 1 times
- **Report the mean and the spread** of the k scores, not one number
- **k = 5 or 10 is standard.** Leave-one-out costs n fits for little gain

Figure from the Week 6 lecture notes

K-fold in scikit-learn: the loop and the shortcuts

```
kf = KFold(n_splits=5, shuffle=True, random_state=0)

scores = []
for train_idx, test_idx in kf.split(X):
    model.fit(X[train_idx], y[train_idx])
    scores.append(model.score(X[test_idx], y[test_idx]))

print(np.mean(scores), np.std(scores))
```

```
scores = cross_val_score(model, X, y, cv=kf, scoring="r2")

detail = cross_validate(model, X, y, cv=kf,
    scoring=["r2", "neg_root_mean_squared_error"],
return_train_score=True)
```

```
rkf = RepeatedKFold(n_splits=5, n_repeats=20, random_state=0)
# 100 fits
scores = cross_val_score(model, X, y, cv=rkf, scoring="r2")
```

Depth-4 tree, 300 samples	R^2
Five fold scores	0.69, 0.67, 0.58, 0.54, 0.79
5-fold mean ± spread	0.65 ± 0.09
Repeated 20 times (100 fits)	0.67 ± 0.07
100 single hold-outs	0.66 ± 0.07

The splitter is the thing to understand. KFold yields a (train rows, test rows) pair per round. Everything else is bookkeeping.

Error metrics come back negated, because scikit-learn maximizes every score. Flip the sign.

Repeat when the fits are cheap. Five scores give a rough spread; a hundred give a reliable one.

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

The rules that come up every time

Rule	Why
Shuffle before cutting folds, unless order matters	KFold without shuffle takes contiguous blocks; rows are usually sorted by something
Stratify for classification (StratifiedKFold)	keeps class proportions equal across folds; essential when a class is rare
Put preprocessing inside a Pipeline	a scaler, imputer, or encoder fitted on all the data has seen every test fold
Report mean and spread, not mean and standard error	fold scores share training data and are not independent
Refit on all the data at the end	the k cross-validation models are throwaway; the deployed model is one fit on everything
Flip the sign on error metrics	scikit-learn maximizes, so RMSE arrives as neg_root_mean_squared_error

Every one of these is a form of the same rule: nothing about the test fold may touch the model before it is scored.

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

The assumption underneath: independent samples

- **Random k-fold assumes samples are independent and identically distributed:** knowing one tells you nothing about another
- **Geophysical data violate this almost everywhere**
- **A random split then puts near-copies of each test sample into the training set.** The model scores by lookup, and the score measures how smooth the field is
- **The inflation is silent.** Nothing in the code warns you

In space

adjacent grid cells and nearby stations are near-duplicates

In time

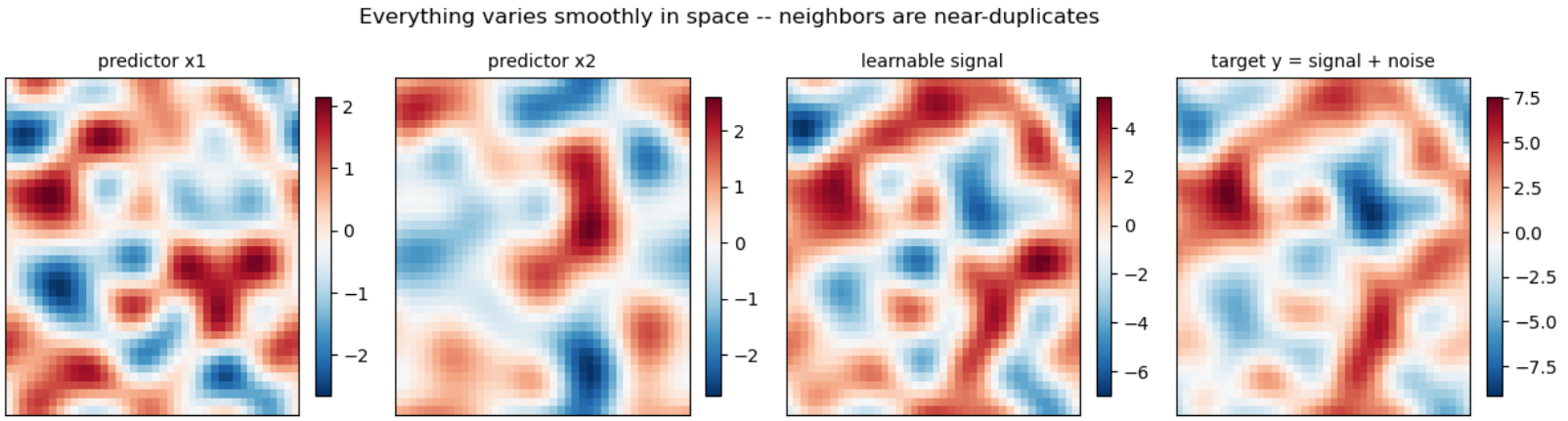
today looks like yesterday; a year of daily rows is far fewer than 365 independent samples

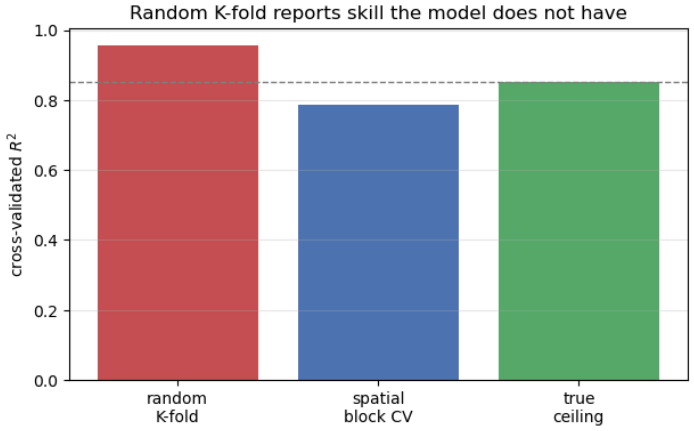
Across the sample

one satellite orbit, one instrument, one model ensemble: shared structure

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2

Space: random folds invent skill

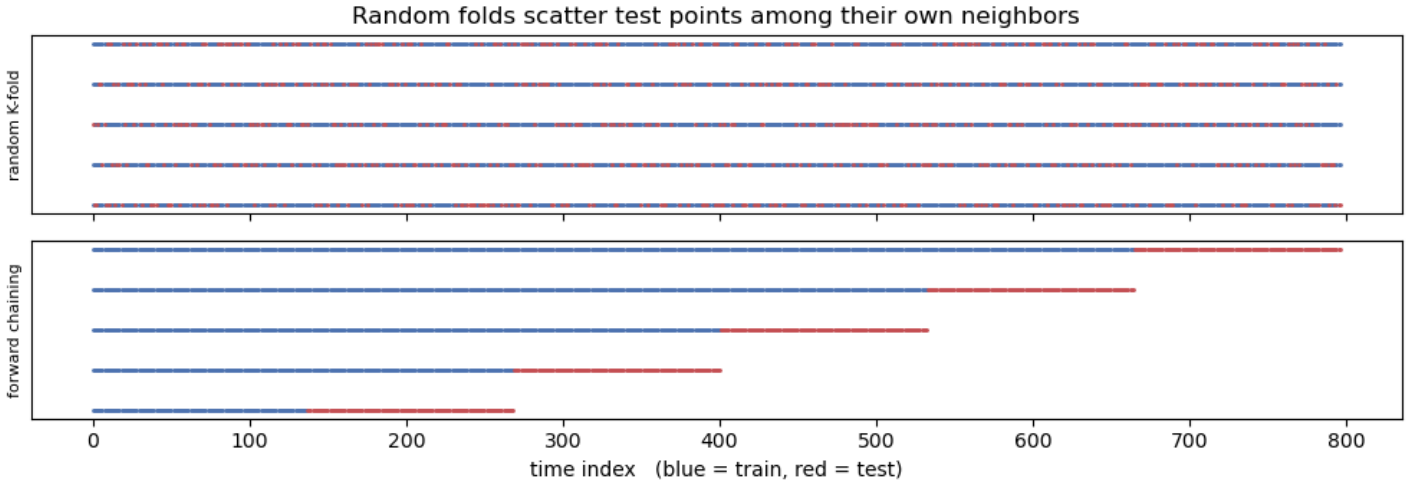




- **The target is signal plus smooth noise that no feature explains,** so the achievable R^2 has a ceiling of 0.85
- **Random 5-fold scores 0.96, above the ceiling.** It is predicting the unpredictable by copying neighbors
- **Hold out contiguous blocks with GroupKFold: 0.79,** just under the ceiling, which is the truth
- **Latitude and longitude as features make it worse.** If they are in, blocked validation is mandatory

Figure from the Week 6 lecture notes

Time: random folds train on the future

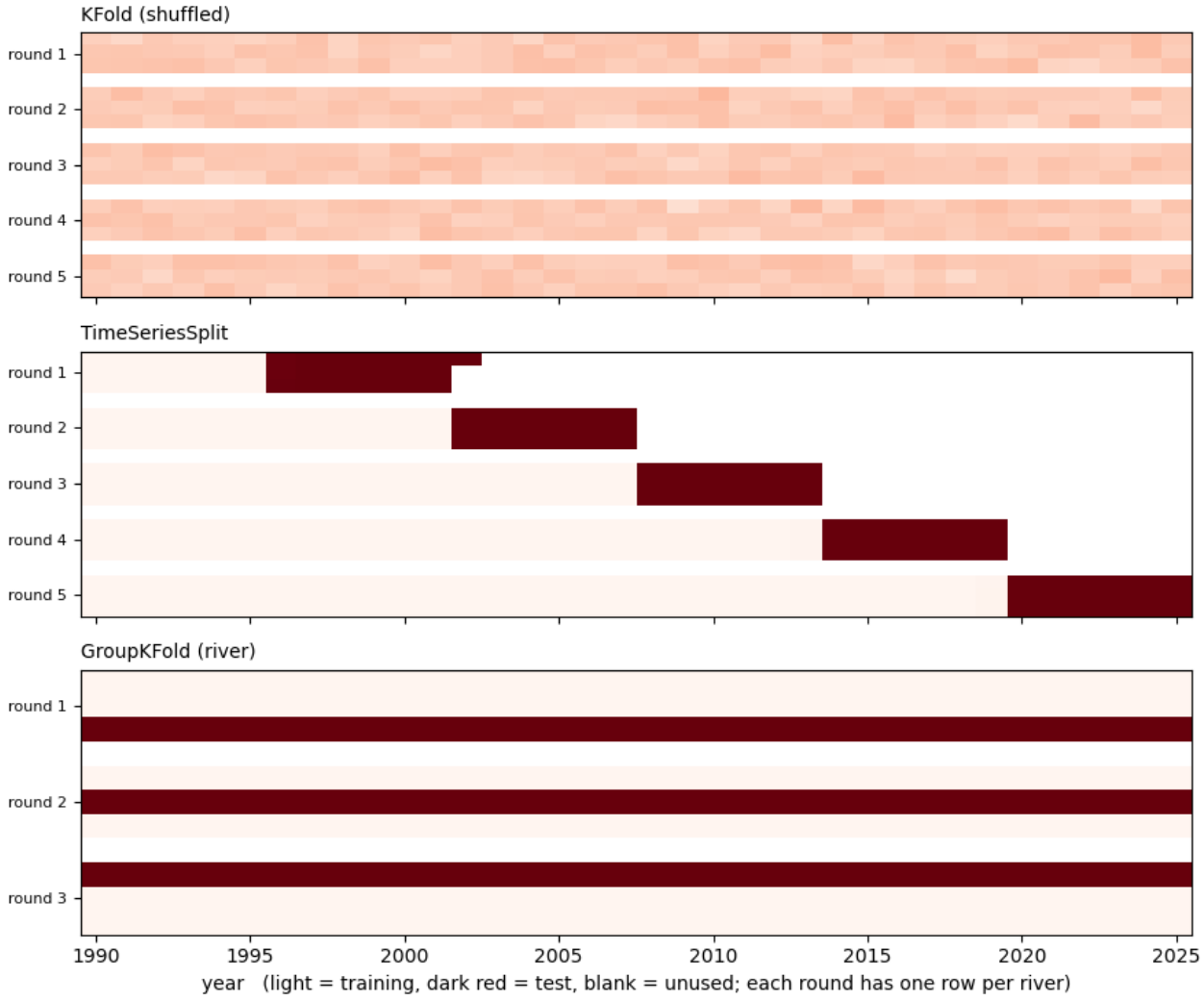


Same forest, same data	R^2
Random 5-fold	0.44
Forward chaining (TimeSeriesSplit)	−0.36
Persistence	0.94

- **A slow unobserved mode makes the target unlearnable from the features**
- **Random folds say 0.44:** a model that seems to work
- **Forward chaining says worse than the mean.** That is the truth, and persistence beats everything
- **The random score was not inflated, it was invented,** from the model recognizing its neighbors in time

Figure from the Week 6 lecture notes

Three splitters, three questions



Split, depth-5 tree	RMSE (log flow)	Persistence
A: random 80/20	0.443	0.447
B: before and after 2018	0.414	0.433
C: hold out the Delaware	0.663	0.606
C: hold out the Willamette	0.761	0.409
C: hold out the Yellowstone	0.648	0.256

- **Each splitter asks a different question:** fill in a missing week, forecast next year, or handle a river never seen
- **Splits A and B agree and sit near persistence.** Split C fails on every river
- **Same data, same code, same tree.** The split is the question

Week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

A split that matches the application finds the flaw

Held-out river, depth-5 tree	Raw log flow	Anomaly from the river's mean	Persistence
Delaware	0.663	0.582	0.606
Willamette	0.761	0.422	0.409
Yellowstone	0.648	0.554	0.256

Reading the table. Same split, same model. The anomaly columns are what the model could learn about rivers in general; the raw columns are what it learned about three particular gauges. A random split would have reported 0.44 for both and never told you the difference.

- **The flaw:** today's flow identifies the gauge. The Willamette runs at ten times the Yellowstone's level, so every threshold learned on two rivers is wrong for the third
- **The fix is in the features:** predict the anomaly from each river's own mean, which a model can carry to a new gauge
- **Two rivers land at persistence.** The Yellowstone does not: a rain-fed river cannot teach the timing of snowmelt
- **Validation is how you find out what the model learned,** not only how well

Week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

Interpolation is not forecasting

Calendar-only tree, Yellowstone	RMSE (log flow)
Random split	0.043
Time split: train before 2018, test after	0.501
Climatology, time split	0.254

The signature to watch for

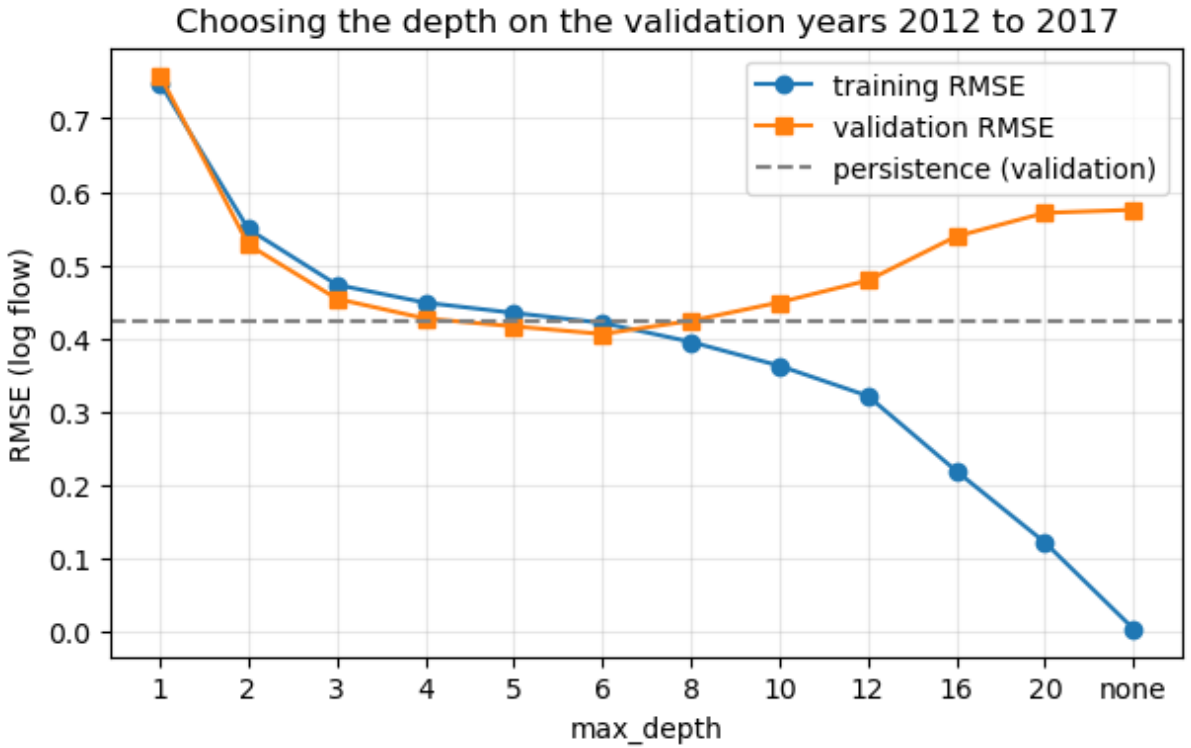
A flexible model, a feature that identifies position in space or time, a random split, and a score that looks too good. The three usual culprits: a timestamp, a station or gauge ID, a coordinate.

- **Features: day of year and the year.** Target: the 15-day mean of log flow. An unlimited tree
- **Random split: nearly perfect.** Each test day's neighbors in the same year are in the training set, on the same smooth curve
- **Time split: twice as bad as climatology.** Asked to forecast a new year, it has nothing to forecast with
- **Any feature that says when or where** lets a flexible model look up its neighbors. Random folds cannot tell lookup from learning

Week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

Choosing hyperparameters without cheating

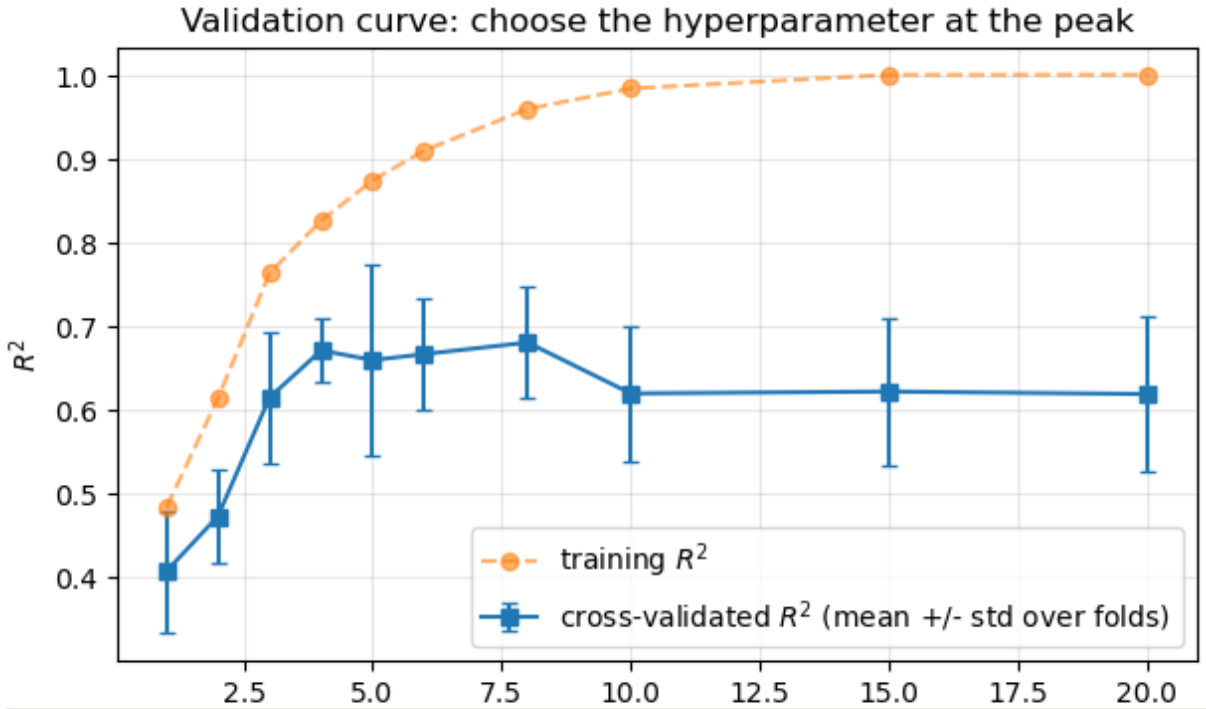
Training	Validation	Test
before 2012	2012 to 2017: chooses the depth	2018 onward: scored once



- **A hyperparameter is a setting fit does not learn:** depth, leaf size, penalty strength, number of trees
- **The rule:** the data that choose it cannot be the data that score the model. Picking the best test score picks a lucky setting
- **The three-way split:** train, validate to choose, test once at the end
- **The validation curve bottoms out at depth 6.** Refit on training plus validation, then one test score: 0.410 against persistence at 0.433

Week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

Tuning by cross-validation: GridSearchCV




```
grid = {"max_depth": [1, 2, 3, 4, 5, 6, 8, 10, 15, 20]}
search = GridSearchCV(DecisionTreeRegressor(), grid,
                      cv=KFold(5, shuffle=True), scoring="r2")
search.fit(X_train, y_train)          # 10 depths x 5 folds
search.best_params_, search.best_score_, search.score(X_test, y_test)
```

- **For each candidate, k-fold on the training set;** keep the best mean, refit, test once
- **The curve plateaus between depths 4 and 8.** The error bars say the choice inside the plateau is noise
- **RandomizedSearchCV** samples combinations; searching depth and leaf size together beat the depth grid
- **best_score_ is optimistic:** the maximum of noisy numbers. Nested CV gave 0.66 against a best_score_ of 0.68

Figure from the Week 6 lecture notes; GridSearchCV after Géron, Ch. 2

Tune with the splitter that matches the application

Temporal data from the notes: random forest, grid over depth and leaf size	Chosen	Cross-validated R^2
Search with shuffled folds	depth 4, leaf 5	0.49
Search with forward chaining	depth 2, leaf 5	−0.26, the truth

River flow from the tutorial: regression tree, grid over depth	Chosen	Test RMSE
Single validation block, 2012 to 2017	depth 6	0.410
GridSearchCV with TimeSeriesSplit	depth 5	0.414
GridSearchCV with shuffled KFold	depth 8	0.412

- **The wrong splitter does not just flatter the score.** On the temporal data it chose a different model, the one best at copying its neighbors
- **On the river data the three choices agree,** because the random and time splits already agreed for those features. You only know that after running the right one
- **GridSearchCV takes any splitter as cv;** GroupKFold takes its groups through fit

Top: figure from the week 6 lecture notes. Bottom: week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

Leakage: when the test set reaches the model

Through preprocessing

A scaler, imputer, encoder, or feature selector fitted on all the data has seen the test set. Put every step in a Pipeline so it is refitted inside each training fold.

Tutorial, ridge regression: scaler on all rows 0.4186, inside the pipeline 0.4186. Invisible here, large for an imputer or encoder, silent either way.

Through a feature

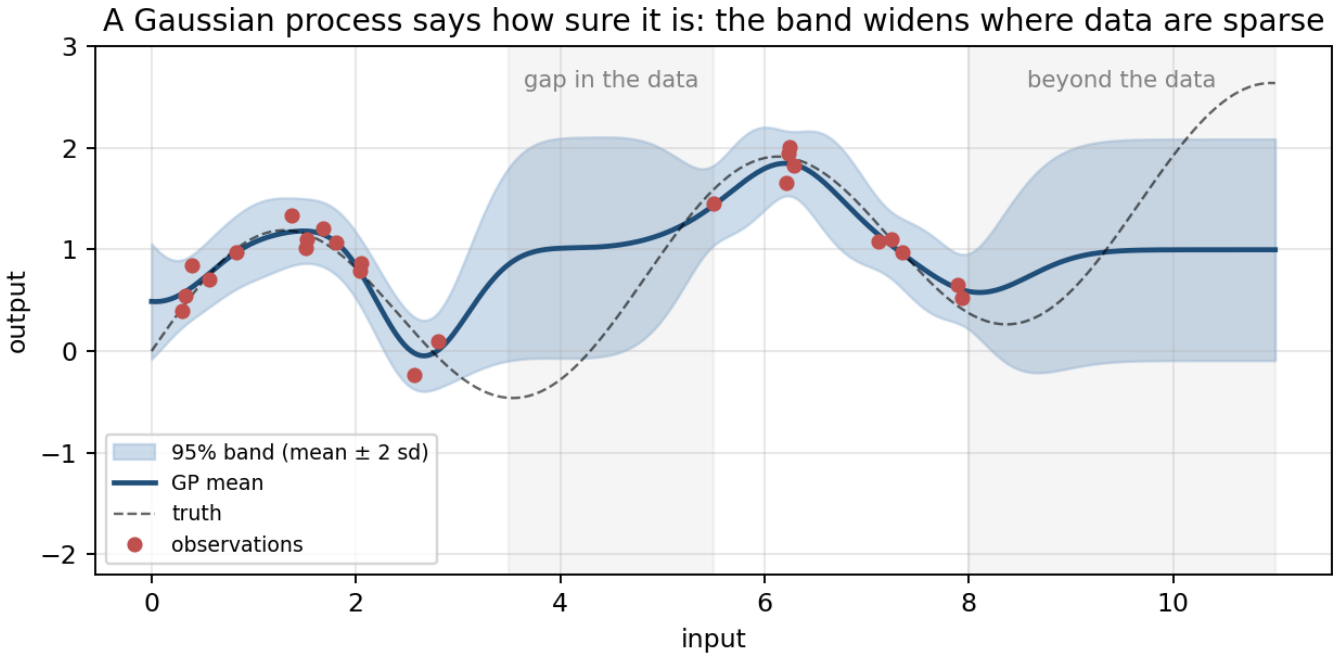
A centered 15-day smoothing window reaches seven days into the future, to the very day being predicted.

Tutorial: legitimate tree 0.410, with the centered feature 0.319, with a trailing window 0.404. Nothing in the code looks wrong.

- **The question to ask of every feature:** would I have this number at the time the prediction is needed?
- **The usual culprits:** centered windows, shift in the wrong direction, future-dated covariates, a column that is the label in disguise
- **If a result seems too good, it probably is.** In this field an unexpectedly high score is more often leakage than a breakthrough

Week 6 tutorial: USGS daily streamflow at the Delaware, Willamette, and Yellowstone gauges, 1990 to 2025

Uncertainty: how wrong might this prediction be?



A model that reports the same uncertainty everywhere, including beyond its data, is not reporting uncertainty at all.

- **Aleatoric:** noise in the system. Measurement error, turbulence. Infinite data would not remove it
- **Epistemic:** ignorance of the model, from finite data. Largest where data are sparse, and reducible by collecting more
- **The two call for different responses:** report the limit of predictability, or go and get more data
- **A Gaussian process reports both:** a mean and a standard deviation at every input. The band fans out where the data end

Gaussian process with an RBF kernel plus a noise term, scikit-learn; following the Week 6 notes on uncertainty

Getting an interval, and checking it

Ensembles

Train several models and read the spread: seeds, bootstrap samples, or different families. A forest's trees are a free ensemble. Caveat: an ensemble only captures the uncertainty it was built to capture, and seeds that agree can be wrong together.

Quantile regression

Fit the 10th, 50th, and 90th percentiles with an asymmetric loss. The spread is an interval that varies with the inputs. For a flood warning, the 95th percentile is the question worth asking.

Conformal prediction

Wrap any model: the 90th percentile of its absolute errors on a held-out calibration set is the half-width of every interval. Guaranteed coverage if calibration and new data are exchangeable, which autocorrelated data are not without blocking.

The check nobody skips twice: when the model says 90%, is it right 90% of the time?

Count the test points inside each nominal interval. For a classifier, plot observed frequency against predicted probability: the reliability diagram, with a calibrated model on the diagonal. Logistic regression is calibrated by construction. Neural networks are usually overconfident, and a correction fitted on held-out data is cheap insurance.

Following the Week 6 notes on uncertainty quantification

Checklist before reporting a result

- **What is the baseline?** Climatology, persistence, or the mean, reported next to the model
- **Is the score one hold-out, or a cross-validated mean with a spread?**
- **Does the split match the application?** New region, hold out regions. Future, hold out the future. New station, hold out stations
- **Were the hyperparameters chosen on data the final score never saw,** with the same splitter?
- **Did any preprocessing or feature see the test set or the future?** Pipelines, and the question: would I have this number when the prediction is needed?
- **Is the reported skill plausible?** A score that seems too good usually is
- **What is the uncertainty, which sources does it include, and is it calibrated?**

Following the Week 6 lecture notes and Géron, Hands-On Machine Learning, 3rd ed., Ch. 2